



Proyecto Jago — Guía Completa Paso a Paso

Asistente virtual con avatar (la mascota jaguar) para la Feria Tecnológica del Saint Margaret School.

Este documento explica **TODO** el proyecto: qué herramientas usamos (todas **gratis**), qué hace cada una, y **cómo se hizo cada paso**, explicado para estudiantes que **apenas inician** en programación.

 Meta del proyecto: un jaguar en una página web que **saluda, escucha, responde hablando, mueve la boca y guarda** con quién conversó. Más adelante se conectará a una IA (Gemini) y se publicará en internet.



Índice

1. [¿Qué necesitas instalar? \(gratis\)](#)
2. [Tabla de TODAS las herramientas y apps](#)
3. [Glosario para principiantes](#)
4. [Cómo está organizado el proyecto \(carpetas\)](#)
5. [Cómo arrancar el proyecto](#)
6. [Paso a paso de lo que YA construimos \(Fases 1–5\)](#)
7. [Lo que sigue \(Fases 6–8\)](#)
8. [Consejos y errores comunes](#)

1) ¿Qué necesitas instalar? (todo gratis)

#	Programa	¿Para qué?	¿Cómo conseguirlo?
1	Google Chrome o Microsoft Edge	Ver la página y usar la voz/micrófono	google.com/chrome
2	Visual Studio Code	Escribir el código (HTML, CSS, JS)	https://code.visualstudio.com
3	Python	Encender un "servidor local" para probar la página	https://www.python.org/downloads (al instalar, marca <code>"Add Python to PATH"</code>)

💡 Python ya venía instalado en la computadora del colegio. Sirve para que el navegador pueda cargar los archivos correctamente (sin él, algunas funciones se bloquean por seguridad).

Extensión recomendada en VS Code (opcional pero muy útil): *Live Server*.

- Abre VS Code → icono de Extensiones (cuadritos a la izquierda) → busca **"Live Server"** → **Install**.
- Luego clic derecho en `index.html` → **"Open with Live Server"** y se abre solo.

2) Tabla de TODAS las herramientas y tecnologías

✓ Ya usadas en el proyecto

Herramienta / Tecnología	¿Qué hace?	¿Gratis?	¿Fácil para estudiantes?	Enlace oficial
HTML	Arma la estructura de la página (textos, botones, cajas)	Sí	Sí, es lo más fácil	developer.mozilla.org/es/docs/Web/HTML
CSS	Da color, diseño y animaciones (boca, respiración)	Sí	Medio	developer.mozilla.org/es/docs/Web/CSS
JavaScript (JS)	Da la "inteligencia": que responda, escuche, hable, guarde datos	Sí	Medio	developer.mozilla.org/es/docs/Web/JavaScript
Web Speech API – SpeechSynthesis	Texto a voz: hace que el jaguar HABLE (viene en el navegador)	Sí	Sí	developer.mozilla.org/en-US/docs/Web/API/SpeechSynthesis
Web Speech API – SpeechRecognition	Voz a texto: hace que el jaguar ESCUCHE el micrófono	Sí	Sí	developer.mozilla.org/en-US/docs/Web/API/SpeechRecognition

SVG	Dibuja la boca (dientes, colmillos, lengua) que se anima	Sí	Medio	developer.mozilla.org/es/docs/Web/SVG
localStorage	Base de datos en el navegador (guarda visitantes sin servidor)	Sí	Medio	developer.mozilla.org/es/docs/Web/API/Window/localStorage
Visual Studio Code	Editor donde escribimos el código	Sí	Sí	code.visualstudio.com
Python (http.server)	Servidor local para probar	Sí	Sí	python.org

 Que pensamos usar (próximas fases)

Herramienta	¿Qué hará?	¿Gratis?	Enlace
Google AI Studio + Gemini API	Darle IA real: que responda preguntas libres (no solo lo guardado)	Sí (plan gratuito)	https://aistudio.google.com
GitHub	Guardar el proyecto en la nube y trabajar en equipo	Sí	https://github.com
GitHub Pages	Publicar la página en internet gratis	Sí	https://pages.github.com
Photopea o GIMP	Editar la imagen del jaguar (recortar boca/ojos si queremos más realismo)	Sí	photopea.com / gimp.org

 Herramientas que comparamos pero NO usamos (y por qué)

Herramienta	Para qué sirve	Por qué NO la usamos
ElevenLabs (voz)	Voz muy realista	Es de pago para uso amplio; la voz del navegador es gratis
Rhubarb Lip Sync	Sincronía de boca perfecta	Necesita un archivo de audio; la voz del navegador no lo da
Live2D / Avatares 3D	Avatares animados profesionales	Mucho trabajo; sería otro personaje, no nuestro jaguar

3) Glosario para principiantes

- **Página web:** un documento que se ve en el navegador. Se hace con HTML + CSS + JS.

- **HTML:** el "esqueleto" (qué hay en la página: títulos, botones, imágenes).
- **CSS:** la "ropa y maquillaje" (colores, tamaños, movimientos/animaciones).
- **JavaScript:** el "cerebro" (qué pasa cuando haces clic, cómo responde, etc.).
- **Navegador:** el programa donde ves páginas (Chrome, Edge).
- **Servidor local:** un programa que "sirve" tus archivos al navegador en tu propia computadora. Lo usamos con Python.
- **API:** una forma de que dos programas se hablen. Ej: la **Web Speech API** deja que tu código le pida al navegador "habla esto".
- **API Key:** una "contraseña" que identifica tu cuenta cuando usas una API en internet (como Gemini). **Nunca se comparte en público.**
- **localStorage:** una "cajita" dentro del navegador donde se guardan datos aunque cierres la página.
- **TTS (Text To Speech):** Texto a Voz.
- **STT (Speech To Text):** Voz a Texto.
- **SVG:** dibujos hechos con código (líneas, círculos) que se pueden animar y nunca se pixelan.

4) Cómo está organizado el proyecto (carpetas)

```

FeriaJaguar/
├── index.html           ← LA PÁGINA (lo que se ve)
├── INICIAR.bat          ← Doble clic para arrancar (Windows)
├── LEEME.txt            ← Guía rápida
├──
├── css/
│   └── estilos.css      ← Diseño, colores y animaciones (boca, respiración)
├── js/
│   ├── avatar.js        ← El "cerebro" dividido por tareas (un equipo por archivo)
│   ├── cerebro.js       ← Habla, mueve la boca, maneja el chat (Equipo 5)
│   ├── cerebro.js       ← Entiende las preguntas y elige respuesta
│   ├── voz.js           ← Escucha por micrófono (Equipo 4)
│   └── base-datos.js    ← Guarda los visitantes (Fase 5)
├──
├── datos/
│   ├── conocimiento.json ← LO QUE SABE el jaguar (Equipo 1 lo llena)
│   └── INFO-QUE-NECESITO.md ← Plantilla para recolectar info de la feria
├──
├── assets/
│   ├── mascota.jpg      ← La imagen del jaguar
│   └── logo.png          ← Logo del colegio
├──
└── documentacion/
    ├── GUIA-EQUIPOS.md   ← Qué hace cada equipo
    └── GUIA-COMPLETA-PASO-A-PASO.md ← (este documento)
  
```

 **Idea clave:** separamos el código en archivos por tarea. Así cada equipo trabaja en SU archivo sin estorbar a los demás.

5) Cómo arrancar el proyecto (2 formas)

Forma A — La más fácil (Windows)

1. Abre la carpeta `FeriaJaguar`.
2. Doble clic en `INICIAR.bat`.
3. Se abre una ventana negra (NO la cierres) y el navegador solo.
4. Si no abre solo, entra a `localhost:8000` en Chrome/Edge.
5. Para cerrar: cierra la ventana negra.

Forma B — Desde VS Code

1. Abre la carpeta en VS Code (**Archivo** → **Abrir carpeta**).
2. Menú **Terminal** → **New Terminal**.
3. Escribe: `python -m http.server 8000` y Enter.
4. Abre `localhost:8000` en el navegador.
5. Para detener: en la terminal, `Ctrl + C`.

 **¿Por qué no abrir el `index.html` con doble clic directo?** Porque el navegador bloquea por seguridad la carga del archivo de conocimiento y el micrófono. Por eso usamos un servidor local (`localhost`).

6) Paso a paso de lo que YA construimos

Fase 1 — El jaguar aparece y HABLA

Qué hicimos: una página con la imagen del jaguar y un botón "Saludar". Al presionarlo, habla en voz alta.

Cómo funciona (concepto): usamos la **Web Speech API** del navegador. En JavaScript:

```
const voz = new SpeechSynthesisUtterance("¡Hola! Soy Jago");
voz.lang = "es-ES";           // idioma español
window.speechSynthesis.speak(voz); // ¡que hable!
```

Para principiantes: `SpeechSynthesisUtterance` es "el texto que quiero que diga". `speak()` lo reproduce.

● Fase 2 — Responde preguntas (chat)

Qué hicimos: un chat. Escribes y el jaguar responde por texto y voz.

Cómo funciona: creamos el archivo `datos/conocimiento.json` (su "cerebro"). Ahí están los saludos, los stands, las preguntas frecuentes. El archivo `js/cerebro.js` **compara** lo que escribes con palabras clave y elige la mejor respuesta.

Concepto clave – "normalizar": antes de comparar, quitamos mayúsculas y acentos para que "¿A qué HORA?" y "a que hora" se entiendan igual.

Para principiantes: es como un buscador: tu pregunta tiene palabras, y buscamos en qué respuesta aparecen esas palabras.

● Fase 3 — Escucha por micrófono

Qué hicimos: un botón 🗣️ "Hablar". Le hablas y el jaguar te entiende y responde.

Cómo funciona: usamos **SpeechRecognition** (también del navegador):

```
const oido = new webkitSpeechRecognition();
oido.lang = "es-ES";
oido.onresult = (e) => {
  const texto = e.results[0][0].transcript; // lo que dijiste, en texto
};
oido.start(); // empieza a escuchar
```

Importante: el micrófono solo funciona en `localhost` (o en https) y pide permiso la primera vez.

● Fase 4 — Mueve la boca + respiración

Qué hicimos: una boca dibujada en **SVG** (con dientes, colmillos y lengua) que se abre y cierra al hablar. Además el jaguar "respira" y se mece suavemente.

Cómo funciona:

- La boca **se sincroniza con el sonido**: empieza a moverse cuando arranca la voz (`onstart`) y se cierra al terminar (`onend`).
- Para que sea **natural**, la boca se mueve suavemente hacia un objetivo que cambia con cada palabra (en vez de saltos bruscos).
- La imagen y la boca van dentro de un mismo contenedor (`.cara`) para **moverse juntas**.

Concepto – animación CSS:

```
@keyframes respirar {
  0%   { transform: translateY(0); }
  50%  { transform: translateY(-3px); } /* sube un poquito */
  100% { transform: translateY(0); }
}
```

🔧 La **posición de la boca** se ajusta con variables en el CSS (`--boca-x` , `--boca-y` , `--boca-ancho` , `--boca-alto`). El botón "Ajustar boca" la muestra con borde para colocarla bien.

● Fase 5 — Base de datos de visitantes

Qué hicimos: al iniciar, pide tu **nombre**. Guarda nombre, hora y todas las preguntas/respuestas. El profesor puede **descargar** todo en Excel (CSV) o JSON.

Cómo funciona: usamos **localStorage** (la cajita del navegador):

```
localStorage.setItem("jago_visitantes", JSON.stringify(lista)); // guardar
const lista = JSON.parse(localStorage.getItem("jago_visitantes")); // leer
```

Ética (Sección G): solo pedimos el **nombre**. Nunca datos sensibles (teléfono, dirección, contraseñas). Los datos quedan en la laptop del kiosco.

7) Lo que sigue (Fases 6–8)

➡ SOON Fase 6 — Integrar a la web de la feria

Mostrar tarjetas de cada stand. Al hacer **clik en una**, el jaguar la **resume en voz alta**.

➡ SOON Fase 7 — Conectar Gemini (IA real)

Para que responda preguntas libres (no solo lo guardado). Pasos:

1. Crear cuenta gratis en **Google AI Studio**.
2. Generar una **API Key**.
3. Conectar la página con Gemini desde `js/ia-gemini.js`.
4. **Seguridad:** la API Key no se sube a internet pública sin protección (lo veremos juntos).

➡ SOON Fase 8 — Publicar en internet

Subir el proyecto a **GitHub** y activar **GitHub Pages** para tener un enlace público gratis.

8) Consejos y errores comunes

-  **Cambié algo y no se ve:** recarga con **Ctrl + F5** (fuerza a recargar el CSS/JS).
 -  **No habla:** haz clic en un botón primero (los navegadores piden un clic antes de reproducir audio). Usa **Chrome o Edge**.
 -  **El micrófono no funciona:** revisa que diste **permiso** y que estás en `localhost` (no con doble clic directo).
 -  **"No pude cargar mi información":** estás abriendo el archivo sin servidor. Usa `INICIAR.bat`
o `python -m http.server`.
 -  **Guardar el código:** en VS Code, **Ctrl + S** después de cada cambio.
 -  **No edites varios archivos a la vez sin guardar:** guarda y prueba paso a paso.
-

✓ Resumen en una frase

Hicimos una **página web gratis** (HTML + CSS + JS) donde un **jaguar habla y escucha** usando la **voz del navegador**, **mueve la boca** con SVG, **guarda visitantes** con localStorage, y pronto tendrá **IA (Gemini)** y estará **publicado** con GitHub Pages.

Documento vivo: se actualizará al avanzar las fases.